

11. Distributed Key-Value Store

Interviewer: Design a distributed key-value store — think Dynamo, Cassandra, Riak. `put(key, value)`, `get(key)`, `delete(key)`. It needs to scale to billions of keys, stay available when nodes fail or the network partitions, and let callers tune how much consistency they're willing to pay latency for. Go.

Candidate: This is the classic "build the database itself" problem — no app-level sharding or caching tricks to hide behind. The hard parts are **how we spread keys across nodes without a full reshuffle every time membership changes**, and **how we keep replicas correct enough without sacrificing availability**. Let me clarify first.

1. Clarifying the requirements

Candidate: A few questions before I size anything: 1. Is the value opaque bytes — no queries, no secondary indexes, no joins? 2. Do we need strict single-key transactions, or is single-key atomicity enough? 3. When the network partitions, do we favor **availability** (keep accepting writes on both sides) or **consistency** (refuse writes rather than risk conflicts)? 4. Is this single-region, or do we need multi-datacenter replication from day one?

Interviewer: Opaque bytes, single-key ops only. Favor availability under a partition — we'd rather accept a write and resolve conflicts later than reject it. Assume multi-region eventually, but start single-region.

Candidate: Good, that settles the CAP question up front — we're building an **AP** system. Requirements:

Functional - `put(key, value)` — write/overwrite. - `get(key)` — read, possibly returning multiple conflicting versions. - `delete(key)` — logical delete (tombstone), not instant physical erase. - No schema, no joins, no cross-key transactions.

Non-functional - **Horizontally scalable** — add nodes for more data/QPS without downtime or a full data reshuffle. - **Highly available** — a `put` should succeed even if some replicas are unreachable. -

Low, predictable latency — single-digit ms for single-key ops. - **Tunable consistency per call** — the caller picks a point on the latency/correctness spectrum. - **No data loss** on node/disk failure; graceful behavior under partition.

2. Estimation

Candidate: Let me size the cluster, because it tells me whether "add a node" needs to be cheap (spoiler: yes).

Assume 1B keys, avg value 1 KB, replication factor $N = 3$.
raw data = $1B * 1KB = 1 TB$
replicated = $1TB * 3 = 3 TB$ spread across the cluster

QPS: 50k reads/sec, 10k writes/sec (read-heavy, but writes matter equally for correctness).

Per-node budget: ~500 GB useful disk, ~5k ops/sec/node.
nodes for storage = $3TB / 500GB \approx 6$ (round up, add headroom)
nodes for QPS = $60k / 5k \approx 12$
-> settle on ~20-30 nodes. Each key's 3 replicas spread load across a subset of them; we add nodes linearly as data/QPS grow.

The real question isn't "how many nodes" – it's "when node #31 joins, how much data has to move, and how much traffic does that reshuffle add on top of live load?" That's the whole game.

3. V1 – the naive version, and why `hash % N` dies

Candidate: Simplest possible V1: one router, `hash(key) % N` picks a node, that node stores the value locally.

```
[ Client ] --put/get(key)--> [ Router: node = hash(key) % N ] --> [ Node_i ]
```

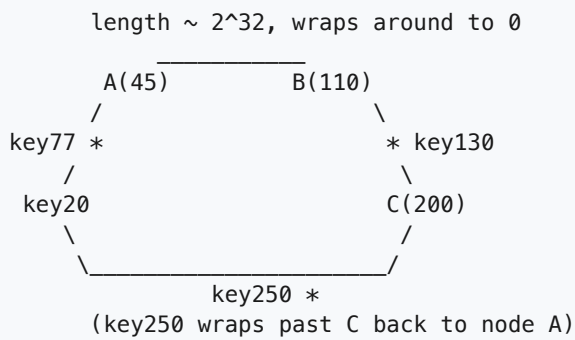
Why it dies the moment N changes: if a node joins or crashes, N changes, and `hash(key) % N` reassigns **almost every key** to a different node. With 1B keys and 3TB of data, that's a near-total reshuffle every time the cluster resizes — exactly what we're trying to avoid, and it happens on the *worst* possible day (a node just died, so we're scrambling while also degraded).

Candidate: So the first real design decision — the one that defines this whole problem — is **how we partition keys so that membership changes move the minimum amount of data**.

4. Partitioning – consistent hashing (the centerpiece)

Interviewer: Right, that's the classic Dynamo problem. Walk me through it.

Candidate: Consistent hashing. Hash both nodes and keys onto the same circular keyspace `[0,  $2^{32}$ )`. A key belongs to the **first node clockwise** from its hash position.



key77 $\rightarrow$ next node clockwise $\rightarrow$ B(110)
 key130 $\rightarrow$ next node clockwise $\rightarrow$ C(200)
 key250 $\rightarrow$ wraps past C, past A's start $\rightarrow$ A(45)

Adding node D at position 160 only steals the slice between B and the old C:

Before: ... B(110) -----[C(200)] ...
 After: ... B(110) -----[D(160)]-[C(200)] ...

Only keys hashing in (110, 160] move – from C to D.
 A's range, B's range, and the rest of C's old range are untouched.
 $\rightarrow \sim 1/N$ of the total keyspace moves, not "almost everything."

Candidate: One point per physical node has a problem though: the arcs are lumpy — one node might own a huge slice, another a sliver, and when a new node joins it only unloads its *one* clockwise neighbor.
Fix: virtual nodes (vnodes) — each physical node claims many points (100-256) scattered around the ring.

Physical node A owns vnodes: A1, A2, A3, ... A100 (scattered around the ring)
 Physical node B owns vnodes: B1, B2, B3, ... B100
 ...
 $\rightarrow$ load spreads evenly across nodes.
 $\rightarrow$ a NEW physical node claims ~ 100 small ranges taken from MANY existing nodes (not just one neighbor) $\rightarrow$ rebalancing work spreads across the whole cluster instead of dogpiling one unlucky neighbor.

Interviewer: Why not just pick a bigger, smarter modulus, or rehash everything on a schedule?

Candidate: A bigger modulus doesn't help — $\% N$ is fundamentally global: change N and *every* key's assignment can change, because the formula depends on the total count. Consistent hashing decouples a key's position from N entirely; a key's assignment only changes if a node crosses *its specific* ring position. Rehashing everything on a schedule just moves the "reshuffle the world" pain to a different clock instead of removing it.

5. Replication and the quorum model

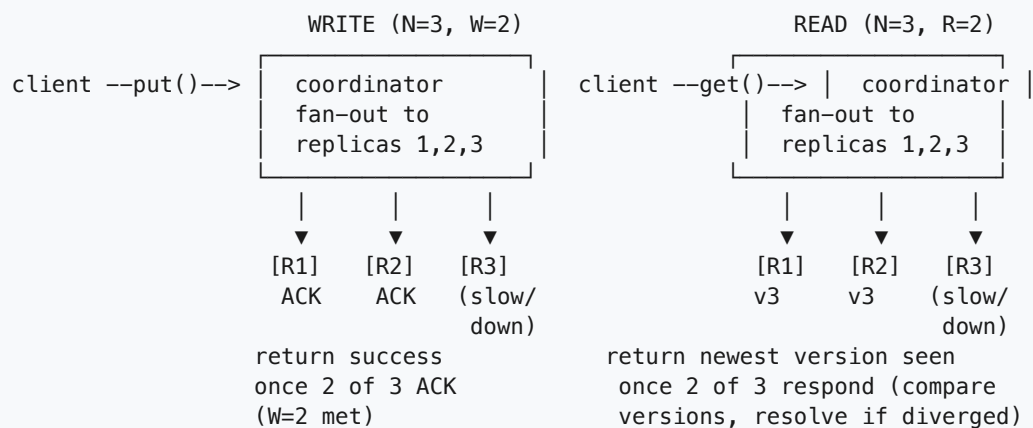
Interviewer: One copy of each key isn't durable. How do you replicate, and how do reads/writes agree on what "success" means?

Candidate: Once `hash(key)` locates a key on the ring, **replicate to the next N-1 nodes clockwise** (N = replication factor, typically 3). The first node hit is the "coordinator" for that key's request.

N=3: key hashes just after B -> owned by C (coordinator for this key).
Replicas also live on the next two distinct physical nodes clockwise.

... B --- [key] --- C (replica 1) --- D (replica 2) --- A (replica 3) ...

Quorum — W, R, N: - **N** = replicas per key. - **W** = replicas that must ACK a write before we call it successful. - **R** = replicas that must respond to a read before we return a result. - **Rule: $W + R > N$** guarantees every read set overlaps the most recent write set on at least one replica.



W=2, R=2, W+R=4 > N=3 -> every read overlaps the latest write on at least one replica -> "read-your-writes"-ish, tolerates 1 dead replica on EITHER path.

Tunable per call:

W=1, R=1 -> fastest, most available, weakest guarantee ("eventual").
W=N, R=1 -> reads cheap & fast; writes must reach everyone (less available).
W=1, R=N -> writes fast; reads check everyone (catches stale replicas, slow).

Why quorum and not "wait for all N"? Waiting for all N replicas on every write means a single slow or dead node blocks every write to its key range — we'd be trading availability for a consistency guarantee we don't need on every call. Quorum lets the caller buy exactly the overlap guarantee they need, per request, while tolerating $N - W$ (or $N - R$) dead replicas without blocking.

6. Multi-region

Interviewer: Now spread this across datacenters. What changes?

Candidate: Replicas are placed **DC-aware** — e.g. N=3 as "2 in DC1, 1 in DC2" (or 1-1-1 across three DCs) — so losing a whole datacenter still leaves a quorum reachable elsewhere.

N=3, rack/DC-aware placement:

DC1: replica A, replica B DC2: replica C

LOCAL_QUORUM: count W/R only against replicas in the CALLER's local DC.

-> normal put()/get() never wait on a cross-WAN round trip.

-> the write still gets replicated to DC2 asynchronously afterward.

Tradeoff: if DC1 fully fails right after a LOCAL_QUORUM write acked locally but before it replicated to DC2, that write can be lost from DC2's point of view until DC1 recovers and anti-entropy (§9) catches it up.

This is exactly Cassandra's LOCAL_QUORUM vs EACH_QUORUM vs QUORUM — the caller picks whether they care more about cross-DC durability or single-DC latency.

7. Latency — where the ms go

Interviewer: A `get` needs to land in single-digit ms. Where does time actually go, and how do you cut it?

Candidate: - **Routing:** client-side, ring-aware routing avoids an extra hop — the client library caches the ring layout and talks *directly* to a replica, instead of bouncing through a random coordinator node first. -

Fan-out, not serial: the coordinator (or smart client) issues the R (or W) requests to replicas **in parallel** with a short per-replica timeout, so one slow replica doesn't stall the whole op — we just need the fastest R (or W) of N to respond. - **Local disk engine:** each node's local store is an **LSM-tree** (RocksDB/LevelDB-style): writes are sequential appends to an in-memory memtable + write-ahead log, so a `put` never waits on a random-write disk seek. Reads may need to check a few SSTables, but a bloom filter + block cache keeps hot reads in the single-digit-ms range. - **Connection reuse:** persistent TCP connections (pooled) between clients/coordinators and nodes avoid paying a TCP/TLS handshake on every op.

Client-side routing (preferred for latency-sensitive callers):

client library knows the ring -> hash(key) locally -> talk DIRECTLY to owning replica(s). One hop.

Any-node coordination (simpler, dumb clients):

client -> any node (round-robin LB) -> that node computes hash(key), forwards to true replicas, collects W/R acks, replies. Two hops.

8. Consistency — what's strong, what's tunable, and why

Interviewer: You said "tunable consistency." Be precise — what's actually guaranteed at each setting?

Candidate: - **Per-key write ordering isn't globally serialized** — there's no transaction log across keys, only per-key convergence. That's fine: this store never promised cross-key consistency. - **W + R > N**

gives a *quorum-consistency* guarantee: any read after an acked write will see that write (or something newer) on at least one replica it consults, so we can always return the newest version by comparing what the R replicas send back. - **$W + R \leq N$** (e.g. $W=1, R=1$) is pure eventual consistency: a read right after a write might hit a replica that hasn't received it yet, and return stale data. That's the latency/consistency knob — cheaper and faster, no overlap guarantee. - **Under a partition, we choose availability.** A put still succeeds on whichever replicas are reachable, even below the ideal W, rather than blocking or erroring out. That makes this an **AP** system in CAP terms — and it means replicas *will* diverge sometimes, which pushes conflict resolution onto the read path (next section) instead of preventing divergence outright.

CAP triangle — pick 2 of 3 during an actual partition:
CP systems (e.g. a config/consensus store) BLOCK writes on the minority side to stay consistent -> lower availability.
AP systems (this store) accept writes on BOTH sides of the partition -> stay available, reconcile divergence after the partition heals.

Conflict resolution when replicas disagree on the same key:

- A) Last-Write-Wins (LWW): attach a timestamp, newest wins.
 - + trivial, no bookkeeping.
 - can SILENTLY discard a legitimate concurrent write if clocks skew.
- B) Vector clocks: each write carries per-node counters, e.g. [A:2, B:1].
 - one version's counters all $\geq$ the other's -> strict descendant, safe to discard the older one.
 - neither dominates (e.g. [A:2,B:1] vs [A:1,B:2]) -> genuinely concurrent -> return BOTH ("siblings") to the app to merge (e.g. union a shopping cart), exactly like Dynamo.

Choice: vector clocks when losing a concurrent write silently is unacceptable; LWW when the data is cache-like and simplicity wins. I'd default to vector clocks for a general-purpose store since that's the harder-but-safer default, and let callers opt into LWW semantics if they don't care.

9. Async — keeping replicas converging without blocking the hot path

Interviewer: Replicas will drift — a node was down during a write, or a read never touches a cold key for months. How do you fix drift without making every request pay for it?

Candidate: None of this runs on the request path — it's all background reconciliation:

Replica down at write time (but W still met by the others):

-> HINTED HANDOFF: a live replica stores the write plus a "hint" for the down node, and forwards it once that node rejoins.

Replicas silently drift (missed write, no crash, nobody noticed):

-> READ REPAIR: on a get(), if the R replicas disagree, return the newest version to the caller AND push it to the stale replica(s) right then, piggybacked on a request that was happening anyway.
-> ANTI-ENTROPY (Merkle trees): each node periodically builds a hash tree over its key ranges and diffs tree hashes with replica peers. Only mismatched subtrees get synced – not a full-dataset diff. This is what catches COLD keys that read repair never touches because nobody reads them.

Node membership changes (join / leave / crash):

-> GOSSIP: nodes periodically exchange "who's alive, and their ring position" with a few random peers. State propagates exponentially fast (like an epidemic) without a central coordinator – every node eventually learns the current ring layout.

GOSSIP – exponential spread, no coordinator

round 1: A tells {B, C}

round 2: {A,B,C} each tell 2 more random peers -> ~6 informed

round 3: ~12 informed ... within $O(\log N)$ rounds, the WHOLE cluster knows node D joined/left, with no single node ever talking to more than a handful of peers.

Why gossip and not a central membership service (e.g. a ZooKeeper-style coordinator)? A central service is a consistency bottleneck and a single dependency that must itself stay available — exactly what an AP store is trying to avoid. Gossip is decentralized, keeps working through partial failures, and its $O(\log N)$ convergence is fast enough for membership, which doesn't need to be instantly consistent everywhere.

10. Caching — where it fits without corrupting correctness

Interviewer: Where would you add a cache in front of this, given the store is already supposed to be fast?

Candidate: Two layers, both careful not to become a second source of truth: - **In-process block cache inside each node's LSM-tree** (already covered in §7) — this is really part of the storage engine, caching hot SSTable blocks in memory so repeated reads of hot keys skip disk entirely. - **An optional client-side or edge cache in front of the whole cluster**, for read-heavy, latency-critical callers who can tolerate staleness — e.g. cache `get(key)` results for a few seconds. This must be **invalidated or simply left to expire on TTL**, never authoritative: any consistency guarantee (quorum, vector clocks) applies to the store, not to a cache sitting in front of it. If a caller needs strict quorum reads, they must bypass this cache.

[Client] --get(key)--> [optional TTL cache] --miss--> [ring-routed R reads]

└ hit: return cached value, KNOWINGLY stale
up to TTL. Never used when caller asked
for R=N / strong-ish reads.

11. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Local storage engine per node	LSM-tree (RocksDB/LevelDB-style)	Writes are sequential appends to a memtable + WAL — no random-write disk seeks, ideal for a write-heavy <code>put</code> path; background compaction keeps read amplification bounded. <i>Not a B-tree (e.g. plain Postgres heap/index)</i> — B-trees do in-place random writes, which get expensive fast at this write volume, and we don't need B-tree's range-query/secondary-index strengths since access is single-key only.
Partitioning scheme	Consistent hashing + vnodes	Only $\sim 1/N$ of keys move on a membership change, and vnodes spread that $\sim 1/N$ across many peers instead of dogpiling one neighbor. <i>Not $hash(key) \% N$</i> — changing N reshuffles almost every key, a massive unnecessary migration triggered by the exact moment (a node dying) when the cluster is already stressed.
Replication / write-success semantics	Quorum (W/R out of N), tunable per call	Lets each caller buy exactly the latency/consistency tradeoff they need, and tolerates $N-W$ (or $N-R$) dead replicas without blocking. <i>Not "wait for all N"</i> — one slow replica would block every write to its range; <i>not "any 1 ack"</i> — that gives no consistency guarantee at all when it matters.
CAP posture	AP (favor availability under partition)	Matches the requirement: <code>put</code> should never be refused just because some replicas are unreachable. <i>Not CP</i> — a CP design (block writes on the minority side) is right for config/coordination data, wrong for a KV store whose whole value proposition is "always accept my write."
Conflict resolution	Vector clocks (LWW as an opt-in simpler mode)	Vector clocks detect true concurrency and surface siblings instead of silently dropping a valid concurrent write — critical when the app can meaningfully merge conflicts (e.g. cart union). <i>Not LWW everywhere</i> — LWW can silently discard a legitimate write on clock skew, which is fine for cache-like data but not a general-purpose guarantee.
Membership propagation	Gossip protocol	Decentralized, keeps converging through partial failures, $O(\log N)$ spread with no single point of failure. <i>Not a central coordinator (e.g. ZooKeeper-style)</i> — that reintroduces a consistency bottleneck and an availability dependency into a system explicitly designed to have neither.
Drift repair	Read repair (hot keys) + Merkle-tree anti-entropy (cold keys)	Read repair fixes drift for free on the read path for keys that ARE being read; anti-entropy is the only thing that ever touches keys nobody reads, using tree-diff to avoid a full

Concern	Choice	Why this, and why not the alternative
		dataset compare. <i>Not "just always read from all N and repair"</i> — that pays the anti-entropy cost on every single read, killing latency for keys that are already fine.
Client routing	Client-side ring-aware routing (smart client), any-node coordinator as fallback	Removes an extra network hop for latency-sensitive internal callers. <i>Not always any-node coordination</i> — that's simpler for dumb/external clients but costs a hop every single call.
Wire protocol	gRPC / binary RPC, TCP, persistent connections	Internal, extremely high-QPS, latency-sensitive protocol — binary framing beats JSON/REST parsing overhead at tens of thousands of ops/sec/node. <i>Not REST/JSON</i> — header + parsing overhead adds up at this volume; a REST facade can still sit in front for external, low-QPS consumers.

Say-it-out-loud justification: *"I partition with consistent hashing plus virtual nodes so a membership change only moves $\sim 1/N$ of the data, spread across the whole cluster instead of dogpiling one neighbor. Each key replicates to N nodes with tunable W/R quorum, so callers trade latency for consistency per call, and $W+R>N$ gives an overlap guarantee when they need it. I favor availability under a partition — writes never block — and resolve the resulting divergence with vector clocks so we never silently drop a real concurrent write. Gossip spreads membership without a central coordinator, and hinted handoff plus read repair plus Merkle-tree anti-entropy keep replicas converging in the background without ever sitting on the hot path. Each node's local engine is an LSM-tree because the access pattern is pure single-key put/get — sequential writes, no wasted B-tree machinery for joins or range scans we'll never do."*

12. Failure modes & graceful degradation

Failure	What happens	Mitigation
One replica down at write time	put still succeeds if W still met by the rest	Hinted handoff stores the write + forwards it when the node returns
Replica silently drifted (missed writes)	Stale reads possible from that replica	Read repair on next get ; Merkle-tree anti-entropy catches cold, unread keys
A whole datacenter fails	Cross-DC replicas of affected keys temporarily reduced	DC-aware placement means a quorum survives in the remaining DC(s); LOCAL_QUORUM writes there resync the failed DC on recovery
Node crashes permanently	Its vnode ranges' replica count drops by one until replaced	Consistent hashing means only that node's $\sim 1/N$ of ranges are under-replicated, not the whole ring; re-replicate onto a new node in the background
Never enough replicas reachable to meet W	put fails/times out	Client can retry, or accept a lower W for that call — the tunable knob in action; never silently under-write without telling the caller
Gossip partition (nodes can't hear each other)	Two sides briefly disagree on ring membership	Gossip is designed to tolerate this — it just reconverges once connectivity returns; writes on both sides still succeed (AP)

Failure	What happens	Mitigation
Hot key (one key gets disproportionate traffic)	vnodes spread <i>data</i> , not one key's <i>load</i>	Client-side or edge caching for that specific key; splitting the value further upstream if it's truly one giant hot record

13. Follow-up curveballs

Interviewer: Why not just increase the modulus or rehash the whole ring on a schedule instead of consistent hashing?

Candidate: The modulus isn't the problem — $\% N$ inherently depends on the *total* count, so changing N changes almost every key's assignment regardless of how big the modulus is. Rehashing on a schedule just moves the "reshuffle everything" pain to a different clock; it doesn't remove the mass-migration event, and it still happens on a schedule instead of exactly when needed. Consistent hashing decouples a key's assignment from N — only keys whose hash falls in the specific range that changed hands actually move.

Interviewer: Two clients write to the same key at nearly the same instant, on opposite sides of a network partition. What actually happens, end to end?

Candidate: Both `put`s succeed locally — the coordinator on each side has enough reachable replicas to meet whatever W was requested (we're AP, so we don't block). Each write's vector clock reflects only what that side has seen, so neither dominates the other. Once the partition heals, anti-entropy/read-repair surfaces both versions as **siblings** on the next `get` — the store doesn't silently pick one, it hands both to the application (unless the caller opted into LWW, in which case whichever timestamp is later wins, with the risk of dropping the other).

Interviewer: How many vnodes per physical node, and why not just one?

Candidate: Commonly 100-256 per node. One vnode per node means a single node's entire range is one contiguous arc — when a new node joins, it can only carve off from its *one* clockwise neighbor, concentrating both the data movement and the resulting load spike on that one unlucky node. Many small vnodes per physical node means a joining node's ranges come from many different existing owners, so the rebalancing cost is spread thin across the whole cluster instead of hammering one peer. More vnodes also smooths load distribution, at the cost of more ring bookkeeping.

Interviewer: A client retries a `put` because it timed out waiting for W acks, even though the first attempt actually landed on 2 of 3 replicas. What stops that from looking like a second concurrent write?

Candidate: The client should reuse the same version context (or attach a client request ID / idempotency token) it used on the first attempt. On retry, the coordinator sees "this is the same causal write, not a new

one" and treats it as a resend rather than a fresh concurrent write that would otherwise produce a spurious sibling. This is exactly the same idempotency concern as any at-least-once system — worth saying explicitly, since it's a common trap: without it, timeouts under load manufacture fake conflicts.

14. Deep-dive: "Why not just modulo hashing — `hash(key) % N`?"

Interviewer: Consistent hashing feels like a lot of machinery — the ring, vnodes, gossip to track it. Why not just keep it simple: `node = hash(key) % N`? Convince me it's actually worse, not just different.

Candidate: It's the right instinct to reach for the simplest thing first, and `% N` is simpler to implement and reason about when `N` is fixed. The problem is entirely about what happens the instant `N` changes — and in a real cluster, `N` changes constantly: nodes get added for capacity, nodes crash, nodes get replaced during maintenance. Let me walk through exactly what breaks.

What happens when a node is added

Say we have 4 nodes and a key currently lives at `hash(key) % 4`. We add a 5th node:

BEFORE (N=4): <code>hash(key) % 4</code>	AFTER (N=5): <code>hash(key) % 5</code>
key A: hash=17 -> 17 % 4 = 1 -> Node 1	17 % 5 = 2 -> Node 2 MOVED
key B: hash=23 -> 23 % 4 = 3 -> Node 3	23 % 5 = 3 -> Node 3 stayed (luck)
key C: hash=31 -> 31 % 4 = 3 -> Node 3	31 % 5 = 1 -> Node 1 MOVED
key D: hash=44 -> 44 % 4 = 0 -> Node 0	44 % 5 = 4 -> Node 4 MOVED
key E: hash=52 -> 52 % 4 = 0 -> Node 0	52 % 5 = 2 -> Node 2 MOVED
4 of 5 keys above moved to a DIFFERENT node just from N: 4 -> 5. There is no structural reason for a key to land near its old node — the whole formula's output depends on the new modulus, so the mapping is essentially re-randomized.	

This isn't a corner case — it's the *general* outcome. Changing the modulus of a hash function has no relationship to the old mapping, so on average `(N-1)/N` of all keys **remap** — with 5 nodes that's ~80% of keys, and it gets worse (asymptotically closer to 100%) as the cluster grows. Every one of those keys is now a **cache miss on the old node and a cold read on the new one** — for our 1B-key, 3TB cluster, that's a near-total data migration triggered by adding *one* node. If the trigger was a node *crashing* instead of a planned add, we're doing this mass reshuffle while already degraded and under retry pressure from clients — the worst possible time for a rebalance storm.

Comparison table

Concern	Modulo hashing (<code>hash(key) % N</code>)	Consistent hashing + vnodes
Keys remapped when N changes	~ (N-1) / N of all keys (nearly all, and worse as N grows)	~ 1/N of all keys — only the range between the changed node and its neighbor(s)

Concern	Modulo hashing ($\text{hash}(\text{key}) \% N$)	Consistent hashing + vnodes
Who absorbs the rebalance	Every node re-fetches/re-caches a large fraction of its keyset simultaneously	Only the joining/leaving node's immediate ring neighbors; with vnodes, spread across the <i>whole</i> cluster in small pieces
Cache behavior on resize	Mass cold-cache event — the "rebalance storm": cache hit rate craters cluster-wide right when load is already abnormal	Localized — most nodes' caches are entirely undisturbed
Load distribution across nodes	Even, <i>if</i> hash is uniform and N is stable	Even with vnodes; lumpy with only one ring-point per node (one node can get a big arc)
Implementation cost	Trivial — one line	Needs a ring structure, vnode bookkeeping, and a way to propagate membership (gossip, §9)
Behavior on node crash (unplanned)	Same mass remap as a planned add/remove — no way to make it gentler	Same small, local remap as a planned change — crash and add cost the same, minimal amount
Predictability of <i>which</i> keys move	Unpredictable — effectively re-randomized by the new modulus	Predictable — exactly the keys whose hash falls in the range that changed hands

Rule of thumb: *Modulo hashing is fine for a fixed-size, rarely-resized shard count decided at deploy time (e.g. a handful of DB shards you almost never change). The moment "nodes join and leave dynamically" is a real operational fact of life — which it always is for a distributed store meant to scale horizontally — the cost of $\% N$'s all-or-nothing remap makes consistent hashing the only sane choice.*

15. Deep-dive: the concurrent-write-under-partition edge case, walked through in detail

Interviewer: Walk me through an actual split-brain write conflict, end to end — not just "vector clocks handle it." Two clients write the *same key* on *opposite sides* of a partition. What does each replica actually hold, second by second, and how do we get back to one answer?

Candidate: This is the sharpest failure mode in an AP system, because it's not a bug — it's the system working *exactly as designed* (favor availability), and design has to own the consequence. Let me lay out the timeline concretely.

Setup

Key `k` normally replicates to 3 nodes: `R1`, `R2`, `R3`. A network partition splits the cluster so `{R1}` can talk to Client X, and `{R2, R3}` can talk to Client Y, but the two groups can't reach each other.

NETWORK PARTITION (X-----X)



Timeline

- t0 Partition begins. R1 isolated from R2/R3.
- t1 Client X: put(k, "red") → coordinator R1
R1 has enough LOCAL replicas to meet whatever W was requested on side A (e.g. W=1 → just itself). We're AP, so R1 ACKs.
R1 stores k = "red", vector clock [R1:1]
- t1' Client Y: put(k, "blue") → coordinator R2 (near-simultaneous, other side)
R2, R3 meet W=2 on side B. ACK returned to Y.
R2/R3 store k = "blue", vector clock [R2:1]
- t2 BOTH writes succeeded. Both clients got a 200 OK. Side A thinks the value is "red"; side B thinks it's "blue". Neither coordinator knows the other write happened — there was no way for it to know, that's the whole meaning of "partitioned."
- t3 Partition heals. R1 and R2/R3 can talk again.
- t4 Anti-entropy (Merkle-tree diff, §9) or the next get() touching k notices R1's version [R1:1]="red" and R2/R3's version [R2:1]="blue" disagree, AND neither vector clock dominates the other ([R1:1] vs [R2:1] — neither is a superset — genuinely concurrent).
- t5 CONFLICT SURFACED: the store cannot know which one is "right" — they're not sequential, they're concurrent. Something has to decide.

Ranked resolution strategies

1. **Last-write-wins (timestamp-based) — simplest, riskiest.** Attach a wall-clock timestamp to each write; whichever is later wins, the other is discarded. Trivial to implement, zero bookkeeping. **The danger:** clock skew. If R1's clock is even a few hundred ms ahead of R2/R3's, "red" (actually written *before* "blue" in real time) can be *stamped* later and silently win — we'd throw away Client Y's write with no error, no signal, nothing. NTP drift, VM clock jumps, and leap-second slew are all real-world ways this bites you. Acceptable when the data is cache-like or "newest observation wins" (e.g. a sensor reading, a last-seen timestamp) — unacceptable when a dropped write is a real loss (money, inventory, a user's data).
2. **Vector clocks / version vectors — safe default, pushes the decision up.** As in §8: each write carries per-replica counters. On conflict, neither [R1:1] nor [R2:1] dominates, so the store **keeps both as siblings** and returns both to the application on the next read, rather than guessing. This never silently drops data — the tradeoff is the app must know how to merge two versions of k (e.g. union, take-max, business-specific logic), and reads occasionally return more than one value.

3. **CRDTs (conflict-free replicated data types) — automatic merge, but shapes the data model.** If the *value itself* is a CRDT (a G-Counter, OR-Set, LWW-Register-with-CRDT- semantics, etc.), the merge function is baked into the data type and is guaranteed to converge without ever asking the application to reconcile siblings by hand — "red" and "blue" written to a last-writer-register-CRDT merge deterministically using embedded metadata, or an OR-Set just unions both adds. The catch: this only works if the value can be modeled as a CRDT in the first place (counters, sets, registers) — an arbitrary opaque blob (this store's actual value type, per our requirements) can't be merged this way unless the caller opts into a specific CRDT-shaped value.
4. **Read-repair + application-level merge — the pragmatic middle ground.** Combine #2 with a merge callback: the store surfaces siblings via vector clocks (never drops data), and read-repair pushes the merged result back out to the stale replica the moment a client resolves it — so the *next* reader sees one converged value instead of re-triggering the same merge. This is exactly Dynamo's actual shopping-cart pattern: siblings on conflict, app-level union, write-back on repair.

Recommended approach

Default to vector clocks with app-level merge (#2 + #4), because this store's values are opaque bytes with no guaranteed structure (per our requirements in §1) — we can't assume CRDT semantics apply, and silently dropping a concurrent write via LWW is not a safe default for a general-purpose store. **Offer LWW as an explicit opt-in** for callers who know their data is cache-like and want the simplicity (and who accept the clock-skew risk). **Offer CRDT value types as a specialized opt-in** (e.g. a counter or set data type) for callers who want automatic, mergeless convergence and can shape their data to fit.

What to say out loud: *"Both writes succeed because we're AP — that's not a bug, that's the contract. The real design decision is what happens when the partition heals and the versions disagree: LWW is simplest but can silently drop a real write under clock skew; vector clocks never lose data but push merging onto the application; CRDTs merge automatically but only work if the value fits a CRDT shape. For a general-purpose opaque- value store I default to vector clocks plus app-level merge, surfaced and healed via read repair — and I'd offer LWW or CRDT value types as explicit, informed opt-ins rather than silent defaults."*

Summary — the mental model

A distributed KV store is **"partition so membership changes move almost nothing, replicate so no single node's failure loses data, and make consistency a dial instead of a fixed setting."** The winning moves: **consistent hashing + vnodes** so a node joining or leaving only reshuffles $\sim 1/N$ of keys, spread across many peers; **N replicas with tunable W/R quorum** so callers trade latency for correctness per call, with $W+R>N$ as the "read-overlaps-write" guarantee; **favor availability under partition** (AP) and push the resulting divergence onto **vector-clock conflict resolution**; and keep replicas converging in the background — **hinted handoff, read repair, Merkle-tree anti-entropy, gossip membership** — so none of that reconciliation cost ever sits on the hot path of a `get` or `put`. Underneath it all, each node is just an LSM-tree doing the one thing this problem actually needs: fast, sequential single-key writes.